

```
/* ===== APP THAI MALIN - PARTAGÉ ENTRE TOUTES LES DESTINATIONS =
Ce fichier est commun à toutes les pages (Bangkok, Phuket, Krabi, Koh Samui, e
Chaque page doit définir un objet window.THAIMALIN_CONFIG AVANT d'inclure ce s
```

```
window.THAIMALIN_CONFIG = {
  citySlug: 'bangkok',          // sert à construire <citySlug>-districts.json
  cityLabel: 'Bangkok',        // utilisé dans les messages d'erreur
  mapCenter: [13.7563, 100.5018],
  mapZoom: 12,
  locationField: 'station',     // 'station' (villes avec métro) ou 'zone' (îl
  locationIcon: '🚇'           // icône affichée à côté du champ ci-dessus ('
};
```

Voir bangkok-premium.html pour un exemple d'intégration complet.

```
=====
```

```
const CONFIG = window.THAIMALIN_CONFIG || {};
const CITY = CONFIG.citySlug || 'ville';
const CITY_LABEL = CONFIG.cityLabel || CITY;
const LOC_FIELD = CONFIG.locationField || 'zone';
const LOC_ICON = CONFIG.locationIcon || '📍';
const MAP_CENTER = CONFIG.mapCenter || [13.7563, 100.5018];
const MAP_ZOOM = CONFIG.mapZoom || 12;
const FAV_KEY = `thaimalin_favorites_${CITY}`;
```

```
/* ===== DONNÉES ===== */
```

```
let districts = [];
let restaurants = [];
```

```
async function loadData(){
  const districtsFile = `${CITY}-districts.json`;
  const restaurantsFile = `${CITY}-restaurants.json`;
  try {
    const [dRes, rRes] = await Promise.all([
      fetch(districtsFile),
      fetch(restaurantsFile)
    ]);
    if(!dRes.ok || !rRes.ok) throw new Error('Fichier JSON introuvable');
    districts = await dRes.json();
    restaurants = await rRes.json();
  } catch(e){
    console.error(`Erreur de chargement des données ${CITY_LABEL} :`, e);
    document.body.insertAdjacentHTML('afterbegin',
```

```

        '<div style="background:#c0392b;color:#fff;padding:14px;text-align:center">
        + `Impossible de charger les données (${districtsFile} / ${restaurantsFile}
        + `Vérifiez qu'elles sont bien à côté de ce fichier HTML et que la page es
    }
    renderDistricts();
    renderRestaurants('all');
    initMap();
}

const catLabel = {street:"Street Food",gastro:"Gastronomique",rooftop:"Rooftop",c
const catColor = {street:"#e8983b",gastro:"#c0392b",rooftop:"#9b59b6",cafe:"#3d8b

/* ===== NAV MOBILE ===== */

const navToggle = document.getElementById('navToggle');
const navLinks = document.getElementById('navLinks');

navToggle.addEventListener('click', ()=>{
    const isOpen = navLinks.classList.toggle('open');
    navToggle.setAttribute('aria-expanded', isOpen ? 'true' : 'false');
});
navLinks.querySelectorAll('a').forEach(link=>{
    link.addEventListener('click', ()=>{
        navLinks.classList.remove('open');
        navToggle.setAttribute('aria-expanded','false');
    });
});

/* ===== QUARTIERS ===== */

const districtGrid = document.getElementById('districtGrid');
const districtSearch = document.getElementById('districtSearch');
const districtFilter = document.getElementById('districtFilter');

function renderDistricts(){
    const q = districtSearch.value.toLowerCase();
    const cat = districtFilter.value;
    districtGrid.innerHTML = districts
        .filter(d => d.name.toLowerCase().includes(q) && (cat==='all' || d.cat===cat))
        .map(d => `
        <article class="tile">
            
            <div class="body">
                <h3>${d.emoji} ${d.name}</h3>
                <p>${d.desc}</p>
                <p><strong>Attractions :</strong> ${d.attractions}</p>
                <p><strong>Restaurant conseillé :</strong> ${d.restaurant}</p>

```

```

        <p><strong>Hôtel conseillé :</strong> ${d.hotel}</p>
        <p class="tip">${d.tip}</p>
    </div>
</article>
`).join('');
}
districtSearch.addEventListener('input', renderDistricts);
districtFilter.addEventListener('change', renderDistricts);

/* ===== FICHES RESTAURANTS ===== */

const restGrid = document.getElementById('restGrid');
const restToolbar = document.getElementById('restToolbar');

let favorites;
try {
    favorites = new Set(JSON.parse(localStorage.getItem(FAV_KEY) || '[]'));
} catch(e) {
    favorites = new Set(); // localStorage indisponible (aperçu isolé, navigation p
}

function saveFavorites(){
    try { localStorage.setItem(FAV_KEY, JSON.stringify([...favorites])); }
    catch(e) { /* stockage indisponible, on continue en mémoire */ }
}

function renderRestaurants(cat){
    const list = restaurants.filter(r => cat==='all' || r.cat===cat);
    restGrid.innerHTML = list.map(r => `
        <article class="tile" data-name="${r.name}">
            
            <div class="body">
                <span class="badge b-${r.cat}">${catLabel[r.cat]}</span>
                <h3>${r.name}</h3>
                <p>${r.desc}</p>
                <p><strong>Prix :</strong> ${r.price} · <strong>Horaires :</strong> ${r.h
                <p><strong>★ Note Thai Malin :</strong> ${r.note} · <strong>${LOC_ICON}<
                <p class="tip">${r.tip}</p>
                <button class="fav-btn" onclick="toggleFav(this, '${r.name.replace(/'/g, "\
            </div>
        </article>
    `).join('');
    syncFavButtons();
}

function toggleFav(btn, name){
    if(favorites.has(name)){ favorites.delete(name); }

```

```

    else{ favorites.add(name); }
    saveFavorites();
    syncFavButtons();
}

function syncFavButtons(){
    document.querySelectorAll('.fav-btn').forEach(btn=>{
        const name = btn.getAttribute('onclick').match(/'([^\']+)'/)[1];
        if(favorites.has(name)){ btn.classList.add('on'); btn.innerHTML='★ Ajouté aux'
        else{ btn.classList.remove('on'); btn.innerHTML='★ Ajouter aux favoris'; }
    });
}

restToolbar.addEventListener('click', e=>{
    if(e.target.tagName!=='BUTTON') return;
    restToolbar.querySelectorAll('button').forEach(b=>b.classList.remove('active'))
    e.target.classList.add('active');
    renderRestaurants(e.target.dataset.cat);
});

/* ===== CARTE INTERACTIVE ===== */

let map, clusterGroup, routeControl, userMarker;

function initMap(){
    if(typeof L === 'undefined') return;

    map = L.map('leafletMap').setView(MAP_CENTER, MAP_ZOOM);

    L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
        maxZoom: 19,
        attribution: '© OpenStreetMap contributors'
    }).addTo(map);

    clusterGroup = L.markerClusterGroup({
        showCoverageOnHover: false,
        spiderfyOnMaxZoom: true,
        zoomToBoundsOnClick: true,
        maxClusterRadius: 60
    });

    renderMapMarkers('all');
    map.addLayer(clusterGroup);

    if(navigator.geolocation){
        navigator.geolocation.getCurrentPosition(pos=>{
            const {latitude, longitude} = pos.coords;

```

```

        userMarker = L.circleMarker([latitude, longitude], {color:'#fff', fillColor
            .addTo(map).bindPopup('Votre position');
    });
}
}

function popupHTML(r) {
    return `
        <strong>${r.name}</strong><br>
        ${catLabel[r.cat]} · ${r.price}<br>
        ★ ${r.note} · ${LOC_ICON} ${r[LOC_FIELD]}<br>
        🍴 ${r.spec}<br><br>
        <button onclick="startRouteTo(${r.lat},${r.lng})">🗺️ Itinéraire</button>
    `;
}

function renderMapMarkers(cat) {
    if(!clusterGroup) return;
    clusterGroup.clearLayers();
    restaurants
        .filter(r => cat==='all' || r.cat===cat)
        .forEach(r=>{
            const icon = L.divIcon({
                className:'',
                html:`<div style="background:${catColor[r.cat]};width:16px;height:16px;bo
                    iconSize:[16,16]
            });
            const marker = L.marker([r.lat, r.lng], {icon});
            marker.bindPopup(popupHTML(r));
            clusterGroup.addLayer(marker);
        });
}

function startRouteTo(lat,lng) {
    if(!navigator.geolocation){ alert("La géolocalisation n'est pas disponible.");
    navigator.geolocation.getCurrentPosition(pos=>{
        if(routeControl){ map.removeControl(routeControl); }
        routeControl = L.Routing.control({
            waypoints: [
                L.latLng(pos.coords.latitude, pos.coords.longitude),
                L.latLng(lat, lng)
            ],
            routeWhileDragging:false,
            show:true,
            addWaypoints:false,
            draggableWaypoints:false,
            fitSelectedRoutes:true

```

```

    }).addTo(map);
}, err=>{
    alert("Impossible d'obtenir votre position : "+err.message);
});
}

document.querySelectorAll('#carte .toolbar button').forEach(btn=>{
    btn.addEventListener('click', ()=>{
        document.querySelectorAll('#carte .toolbar button').forEach(b=>b.classList.re
        btn.classList.add('active');
        renderMapMarkers(btn.dataset.mapcat);
    });
});

document.addEventListener('DOMContentLoaded', loadData);
window.addEventListener('resize', ()=>{ if(map){ map.invalidateSize(); } });

```